

AI LAB EXPERIMENTS

T. Subahan

192511121

Experiment 1:

Aim:

To solve the **8-Puzzle Problem** using the **Breadth-First Search (BFS)** algorithm in Python and find the shortest sequence of moves from the initial state to the goal state.

Algorithm:

1. Define the initial state and goal state of the puzzle.
2. Store the possible movements of the blank tile (up, down, left, right).
3. Use a queue to perform Breadth-First Search (BFS).
4. Remove the first state from the queue.
5. If the current state matches the goal state, stop and display the solution path.
6. Otherwise, generate all valid neighboring states by moving the blank tile.
7. Add unvisited neighboring states to the queue.
8. Repeat the process until the goal state is reached or no solution exists.

Program :

```
from collections import deque
```

```
# Function to find the position of blank tile (0)
```

```
def find_blank(state):
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if state[i][j] == 0:
```

```
                return i, j
```

```

# Function to generate neighboring states
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)

    directions = [(-1,0), (1,0), (0,-1), (0,1)]

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

# BFS Algorithm
def bfs(start, goal):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        current, path = queue.popleft()

        if current == goal:
            return path + [current]

```

```
if current not in visited:
```

```
    visited.add(current)
```

```
    for neighbor in get_neighbors(current):
```

```
        queue.append((neighbor, path + [current]))
```

```
return None
```

```
# Initial and Goal States
```

```
start = (
```

```
    (1, 2, 3),
```

```
    (4, 0, 6),
```

```
    (7, 5, 8)
```

```
)
```

```
goal = (
```

```
    (1, 2, 3),
```

```
    (4, 5, 6),
```

```
    (7, 8, 0)
```

```
)
```

```
solution = bfs(start, goal)
```

```
if solution:
```

```
    print("Solution found in", len(solution)-1, "moves:\n")
```

```
    for step in solution:
```

```
        for row in step:
```

```
            print(row)
```

```
        print()
    else:
        print("No solution exists.")
```

Output :



The screenshot shows a code editor with a file named 'main.py'. The code implements a Breadth-First Search (BFS) algorithm for the 8-Puzzle problem. It defines a goal state, a set of moves, and functions to find the blank tile and convert the state to a tuple. The output window shows the solution found: [1, 2, 3], [4, 0, 6], [7, 5, 8].

```
main.py
1 from collections import deque
2
3 goal = [[1, 2, 3],
4         [4, 5, 6],
5         [7, 8, 0]]
6
7 moves = [(-1,0),(1,0),(0,-1),(0,1)]
8
9 def find_blank(state):
10     for i in range(3):
11         for j in range(3):
12             if state[i][j] == 0:
13                 return i, j
14
15 def to_tuple(state):
16     return tuple(tuple(row) for row in state)
17
```

Output

Solution found:

```
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
```

=== Code Execution Successful ===

Result :

The 8-Puzzle Problem was successfully solved using the Breadth-First Search (BFS) algorithm in Python. The program found the shortest sequence of moves required to transform the initial state into the goal state.

Experiment 2:

Aim:

To solve the **8-Queen Problem** using the **Backtracking Algorithm** in Python, such that no two queens attack each other on an 8×8 chessboard.

Algorithm:

1. Start placing queens from the first row.
2. For each row, try placing a queen in every column.
3. Check whether the position is safe:
 - No other queen is in the same column.
 - No other queen is on the left diagonal.
 - No other queen is on the right diagonal.
4. If the position is safe, place the queen and move to the next row.
5. If no safe position exists, backtrack by removing the previously placed queen.
6. Repeat the process until all 8 queens are placed successfully.
7. Display the final chessboard arrangement.

N = 8

Program :

Function to print the chessboard

```
def print_board(board):
```

```
    for row in board:
```

```
        print(" ".join("Q" if cell else "." for cell in row))
```

Check if a queen can be placed safely

```
def is_safe(board, row, col):
```

```
    # Check column
```

```
    for i in range(row):
```

```
        if board[i][col]:
```

```
            return False
```

```
# Check left diagonal
```

```
i, j = row - 1, col - 1
```

```
while i >= 0 and j >= 0:
```

```
    if board[i][j]:
```

```
        return False
```

```
    i -= 1
```

```
    j -= 1
```

```
# Check right diagonal
```

```
i, j = row - 1, col + 1
```

```
while i >= 0 and j < N:
```

```
    if board[i][j]:
```

```
        return False
```

```
    i -= 1
```

```
    j += 1
```

```
return True
```

```
# Backtracking function
```

```
def solve(board, row):
```

```
    if row == N:
```

```
        return True
```

```
    for col in range(N):
```

```
        if is_safe(board, row, col):
```

```
            board[row][col] = 1
```

```
            if solve(board, row + 1):
```

```
                return True
```

```
board[row][col] = 0
```

```
return False
```

```
# Main Program
```

```
board = [[0] * N for _ in range(N)]
```

```
if solve(board, 0):
```

```
    print("Solution Exists:\n")
```

```
    print_board(board)
```

```
else:
```

```
    print("No Solution Exists.")
```

output:

```
C:\Users\Subahan> from collections import deque.py / ...
1  N = 8
2
3  # Function to print the chessboard
4  def print_board(board):
5      for row in board:
6          print(" ".join("Q" if cell else "." for cell in row))
7
8  # Check if a queen can be placed safely
9  def is_safe(board, row, col):
10     # Check column
11     for i in range(row):
12         if board[i][col]:
13             return False
14
15     # Check left diagonal
16     i, j = row - 1, col - 1
17     while i >= 0 and j >= 0:
18         if board[i][j]:
19             return False
20         i -= 1
21         j -= 1
22
23     # Check right diagonal
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + - [] [] ... []

```
e' 'c:\Users\Subahan\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '57814'
-' 'c:\Users\Subahan\from collections import deque.py'
Solution Exists:

Q . . . . .
. . . Q . .
. . . . . Q
. . . . Q .
. Q . . . .
. . . . . Q
. Q . . . .
. . Q . . .
PS C:\Users\Subahan>
```

Result:

The 8-Queen Problem was successfully solved using the Backtracking Algorithm in Python. The program placed all eight queens on the chessboard such that no two queens attack each other.

Experiment 3:

Aim :

To solve the **Water Jug Problem** using the **Breadth-First Search (BFS)** algorithm in Python and obtain the required quantity of water.

Algorithm :

1. Define the capacities of the two water jugs.
2. Specify the target amount of water to be measured.
3. Represent each state as (Jug1, Jug2).
4. Start from the initial state (0, 0).
5. Use Breadth-First Search (BFS) to explore all possible states.
6. From each state, generate the next possible states by:
 - Filling Jug 1.
 - Filling Jug 2.
 - Emptying Jug 1.
 - Emptying Jug 2.
 - Pouring water from Jug 1 to Jug 2.
 - Pouring water from Jug 2 to Jug 1.
7. If the target amount is found in either jug, display the sequence of states.
8. If no solution exists, print an appropriate message.

Program :

```
from collections import deque

def water_jug_bfs(cap1, cap2, target):
    visited = set()
    queue = deque([((0, 0), [])])

    while queue:
        (jug1, jug2), path = queue.popleft()
```

```
if (jug1, jug2) in visited:
```

```
    continue
```

```
visited.add((jug1, jug2))
```

```
path = path + [(jug1, jug2)]
```

```
if jug1 == target or jug2 == target:
```

```
    return path
```

```
next_states = [
```

```
    (cap1, jug2),          # Fill Jug 1
```

```
    (jug1, cap2),          # Fill Jug 2
```

```
    (0, jug2),             # Empty Jug 1
```

```
    (jug1, 0),             # Empty Jug 2
```

```
    (jug1 - min(jug1, cap2-jug2),
```

```
        jug2 + min(jug1, cap2-jug2)), # Pour Jug 1 -> Jug 2
```

```
    (jug1 + min(jug2, cap1-jug1),
```

```
        jug2 - min(jug2, cap1-jug1)) # Pour Jug 2 -> Jug 1
```

```
]
```

```
for state in next_states:
```

```
    if state not in visited:
```

```
        queue.append((state, path))
```

```
return None
```

```
# Main Program
```

```
jug1_capacity = 4
```

```
jug2_capacity = 3
```

```
target = 2
```

```
solution = water_jug_bfs(jug1_capacity, jug2_capacity, target)
```

```
if solution:
```

```
    print("Solution Path:")
```

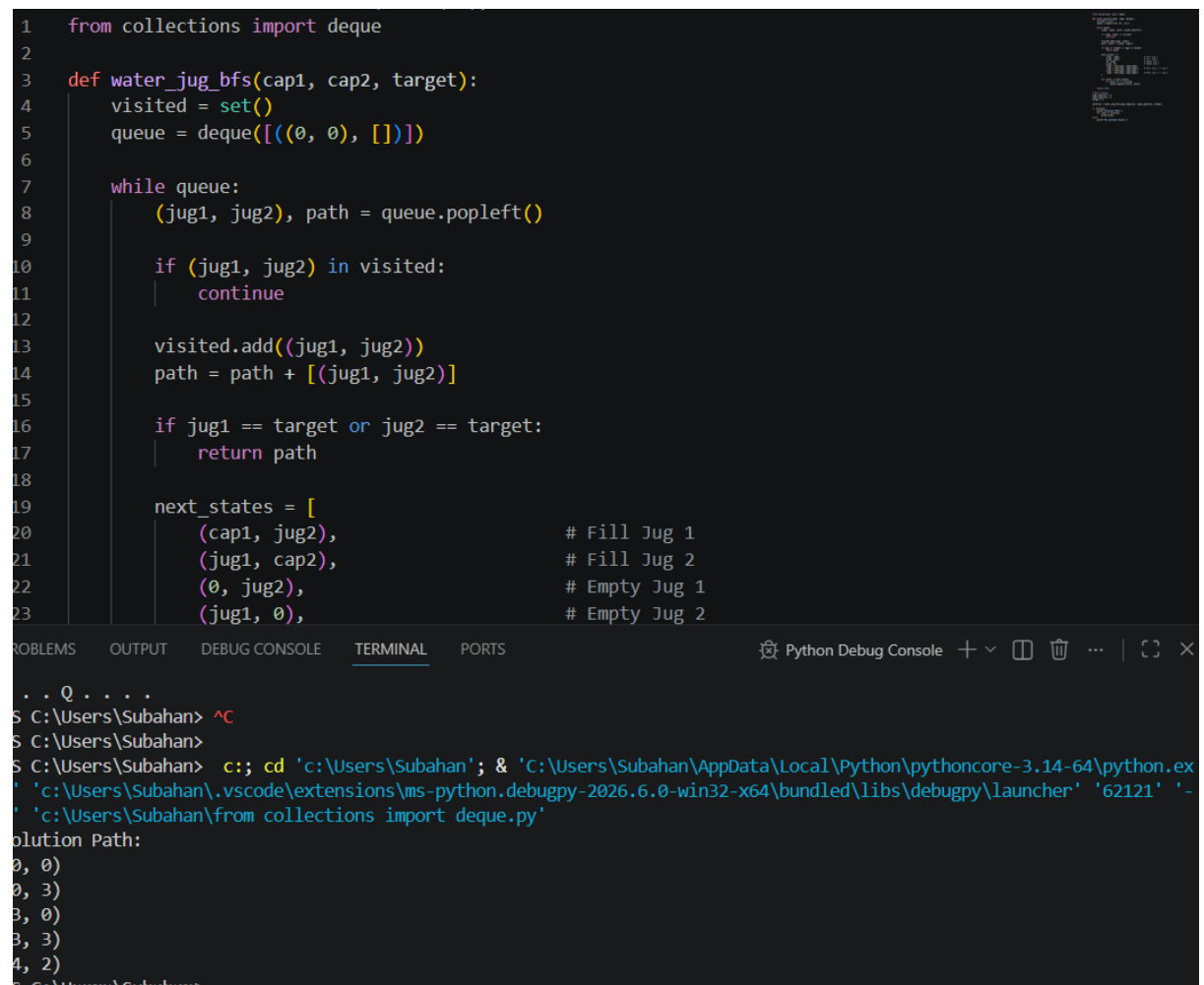
```
    for step in solution:
```

```
        print(step)
```

```
else:
```

```
    print("No solution exists.")
```

Output :



```
1  from collections import deque
2
3  def water_jug_bfs(cap1, cap2, target):
4      visited = set()
5      queue = deque([(0, 0), []])
6
7      while queue:
8          (jug1, jug2), path = queue.popleft()
9
10         if (jug1, jug2) in visited:
11             continue
12
13         visited.add((jug1, jug2))
14         path = path + [(jug1, jug2)]
15
16         if jug1 == target or jug2 == target:
17             return path
18
19         next_states = [
20             (cap1, jug2),          # Fill Jug 1
21             (jug1, cap2),          # Fill Jug 2
22             (0, jug2),             # Empty Jug 1
23             (jug1, 0),             # Empty Jug 2
24
25         ]
26
27         for next_state in next_states:
28             queue.append((next_state[0], next_state[1], path))
29
30     return None
31
32 if __name__ == '__main__':
33     jug1_capacity = 4
34     jug2_capacity = 3
35     target = 2
36     solution = water_jug_bfs(jug1_capacity, jug2_capacity, target)
37
38     if solution:
39         print("Solution Path:")
40         for step in solution:
41             print(step)
42     else:
43         print("No solution exists.")
```

..Q....

S C:\Users\Subahan> ^C

S C:\Users\Subahan>

S C:\Users\Subahan> c:: cd 'c:\Users\Subahan'; & 'C:\Users\Subahan\AppData\Local\Python\pythoncore-3.14-64\python.exe

' 'c:\Users\Subahan\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62121' '-

' 'c:\Users\Subahan\from collections import deque.py'

olution Path:

0, 0)

0, 3)

3, 0)

3, 3)

4, 2)

S C:\Users\Subahan>

Result: The Water Jug Problem was successfully solved using the Breadth-First Search (BFS) algorithm in Python. The program found the sequence of operations required to measure the target quantity of water.

Experiment 4:

Aim:

To solve a **Cryptarithmic Problem** using Python by assigning unique digits to letters such that the given arithmetic equation is satisfied.

Algorithm:

1. Identify all unique letters in the cryptarithmic expression.
2. Assign a unique digit (0–9) to each letter.
3. Ensure that no two letters are assigned the same digit.
4. Ensure that the leading letters are not assigned the digit 0.
5. Check whether the arithmetic equation is satisfied.
6. If a valid assignment is found, display the solution.
7. Otherwise, report that no solution exists.

Program :

```
from itertools import permutations
```

```
letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
```

```
digits = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

```
for perm in permutations(digits, len(letters)):
```

```
    d = dict(zip(letters, perm))
```

```
    # Leading letters cannot be zero
```

```
    if d['S'] == 0 or d['M'] == 0:
```

```
        continue
```

```
    SEND = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
```

```
    MORE = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
```

```
    MONEY = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']
```

```

if SEND + MORE == MONEY:

    print("Solution Found:")

    print("SEND =", SEND)

    print("MORE =", MORE)

    print("MONEY =", MONEY)

    print("\nLetter Assignments:")

    for k, v in d.items():

        print(k, "=", v)

    break

```

Output :

The screenshot shows a Python IDE with a dark theme. The editor displays a script that uses the `itertools.permutations` module to find a solution to the cryptarithm $SEND + MORE = MONEY$. The script defines sets of letters and digits, iterates through permutations, and checks for a valid solution where the sum of the numbers represented by the words equals the number represented by the result. Once a solution is found, it prints the values of SEND, MORE, and MONEY, followed by the letter assignments.

```

1 from itertools import permutations
2
3 letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
4 digits = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
5
6 for perm in permutations(digits, len(letters)):
7     d = dict(zip(letters, perm))
8
9     # Leading letters cannot be zero
10    if d['S'] == 0 or d['M'] == 0:
11        continue
12
13    SEND = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
14    MORE = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
15    MONEY = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']
16
17    if SEND + MORE == MONEY:
18        print("Solution Found:")
19        print("SEND =", SEND)
20        print("MORE =", MORE)
21        print("MONEY =", MONEY)
22        print("\nLetter Assignments:")
23        for k, v in d.items():

```

The terminal output shows the solution found:

```

MORE = 1085
MONEY = 10652

Letter Assignments:
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2
PS C:\Users\Subahan>

```

Result: The Cryptarithmic Problem ($SEND + MORE = MONEY$) was successfully solved using Python. The program assigned unique digits to each letter such that the arithmetic equation was satisfied.

Experiment 5:

Aim:

To implement the **Missionaries and Cannibals Problem** using Python and find a valid sequence of moves that safely transfers all missionaries and cannibals from the left bank of the river to the right bank.

Algorithm:

1. Start with 3 missionaries and 3 cannibals on the left bank.
2. Represent each state as (Missionaries Left, Cannibals Left, Boat Side).
3. Generate all possible boat moves:
 - (2 Missionaries)
 - (2 Cannibals)
 - (1 Missionary and 1 Cannibal)
 - (1 Missionary)
 - (1 Cannibal)
4. Check whether each new state is valid:
 - Missionaries should never be outnumbered by cannibals on either bank (unless there are no missionaries on that bank).
5. Use **Breadth-First Search (BFS)** to explore valid states.
6. Continue until the goal state (0, 0, Right) is reached.
7. Display the sequence of states from the initial state to the goal state.
8. Stop the program.

Program:

```
from collections import deque
```

```
# Check if a state is valid
```

```
def is_valid(m, c):
```

```
    if m < 0 or c < 0 or m > 3 or c > 3:
```

```
        return False
```

```
    if (m > 0 and m < c):
```

```
        return False
```

```
if ((3 - m) > 0 and (3 - m) < (3 - c)):
```

```
    return False
```

```
return True
```

```
# BFS to solve the problem
```

```
def missionaries_cannibals():
```

```
    start = (3, 3, 'L')
```

```
    goal = (0, 0, 'R')
```

```
    moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]
```

```
    queue = deque([(start, [start])])
```

```
    visited = set()
```

```
    while queue:
```

```
        state, path = queue.popleft()
```

```
        if state == goal:
```

```
            return path
```

```
        if state in visited:
```

```
            continue
```

```
        visited.add(state)
```

```
        m, c, boat = state
```

```
        for dm, dc in moves:
```

```
            if boat == 'L':
```

```
        new_state = (m-dm, c-dc, 'R')
    else:
        new_state = (m+dm, c+dc, 'L')

    nm, nc, nb = new_state

    if is_valid(nm, nc) and new_state not in visited:
        queue.append((new_state, path + [new_state]))

    return None

# Display solution
solution = missionaries_cannibals()

print("Solution Path:")
for state in solution:
    print(state)
```

output:

```
1  from collections import deque
2
3  # Check if a state is valid
4  def is_valid(m, c):
5      if m < 0 or c < 0 or m > 3 or c > 3:
6          return False
7      if (m > 0 and m < c):
8          return False
9      if ((3 - m) > 0 and (3 - m) < (3 - c)):
10         return False
11     return True
12
13 # BFS to solve the problem
14 def missionaries_cannibals():
15     start = (3, 3, 'L')
16     goal = (0, 0, 'R')
17
18     moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]
19
20     queue = deque([(start, [start])])
21     visited = set()
22
23     while queue:
```

ROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
3, 3, 'L')
3, 1, 'R')
3, 2, 'L')
3, 0, 'R')
3, 1, 'L')
1, 1, 'R')
2, 2, 'L')
0, 2, 'R')
0, 3, 'L')
0, 1, 'R')
1, 1, 'L')
0, 0, 'R')
```

S C:\Users\Subahan> █

Result:

The **Missionaries and Cannibals Problem** was successfully solved using the **Breadth-First Search (BFS)** algorithm. The program found a valid sequence of moves that safely transported all three missionaries and three cannibals across the river without ever leaving missionaries outnumbered by cannibals on either bank.

Experiment 6:

Aim:

To implement the **Vacuum Cleaner Problem** using Python and simulate a simple intelligent agent that cleans all dirty rooms.

Algorithm:

1. Start the program.
2. Initialize the environment with two rooms (A and B), each marked as **Clean** or **Dirty**.
3. Set the initial position of the vacuum cleaner.
4. Check the current room:
 - If the room is **Dirty**, clean it.
 - If the room is **Clean**, move to the next room.
5. Repeat the process until both rooms are clean.
6. Display the final status of all rooms.
7. Stop the program.

Program:

```
# Vacuum Cleaner Problem
```

```
def vacuum_cleaner(room_a, room_b, location):
```

```
    rooms = {"A": room_a, "B": room_b}
```

```
    while True:
```

```
        if rooms[location] == "Dirty":
```

```
            print(f"Room {location} is Dirty.")
```

```
            print(f"Cleaning Room {location}...")
```

```
            rooms[location] = "Clean"
```

```
        else:
```

```
            print(f"Room {location} is already Clean.")
```

```
    if location == "A":
```

```
        location = "B"
    else:
        location = "A"

    if rooms["A"] == "Clean" and rooms["B"] == "Clean":
        break

    print("\nFinal Status:")
    print("Room A:", rooms["A"])
    print("Room B:", rooms["B"])

# Example
vacuum_cleaner("Dirty", "Dirty", "A")
```

output:

```
C: > Users > Subahan > from collections import deque.py > vacuum_cleaner
1  # Vacuum Cleaner Problem
2
3  def vacuum_cleaner(room_a, room_b, location):
4      rooms = {"A": room_a, "B": room_b}
5
6      while True:
7          if rooms[location] == "Dirty":
8              print(f"Room {location} is Dirty.")
9              print(f"Cleaning Room {location}...")
10             rooms[location] = "Clean"
11         else:
12             print(f"Room {location} is already Clean.")
13
14         if location == "A":
15             location = "B"
16         else:
17             location = "A"
18
19         if rooms["A"] == "Clean" and rooms["B"] == "Clean":
20             break
21
22     print("\nFinal Status:")
23     print("Room A:", rooms["A"])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Subahan> & 'C:\Users\Subahan\AppData\Local\Python\pythoncore-
gpy\launcher' '59252' '--' 'c:\Users\Subahan\from collections import deque
Room A is Dirty.
Cleaning Room A...
Room B is Dirty.
Cleaning Room B...

Final Status:
Room A: Clean
Room B: Clean
PS C:\Users\Subahan>
```

Result:

The vacuum cleaner agent successfully cleaned all dirty rooms and changed their status from **Dirty** to **Clean**, demonstrating the working of a simple intelligent agent in an AI environment.

Experiment 7 :

Aim:

To implement the **Breadth-First Search (BFS)** algorithm in Python and traverse all the vertices of a graph in breadth-first order.

Algorithm:

1. Start the program.
2. Represent the graph using an adjacency list.
3. Select the starting vertex.
4. Create an empty queue and a set to store visited vertices.
5. Insert the starting vertex into the queue and mark it as visited.
6. Repeat until the queue is empty:
 - Remove the front vertex from the queue.
 - Display the vertex.
 - Visit all its unvisited adjacent vertices.
 - Mark each adjacent vertex as visited and add it to the queue.
7. Stop when all reachable vertices have been visited.
8. End the program.

Program:

```
from collections import deque

# Graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
```

```
'F': []  
}
```

```
def bfs(graph, start):  
    visited = set()  
    queue = deque([start])  
  
    visited.add(start)  
  
    while queue:  
        vertex = queue.popleft()  
        print(vertex, end=" ")  
  
        for neighbor in graph[vertex]:  
            if neighbor not in visited:  
                visited.add(neighbor)  
                queue.append(neighbor)  
  
# Driver code  
print("BFS Traversal:")  
bfs(graph, 'A')
```

output:

```
1
2  from collections import deque
3
4  # Graph represented as an adjacency list
5  graph = {
6      'A': ['B', 'C'],
7      'B': ['D', 'E'],
8      'C': ['F'],
9      'D': [],
10     'E': ['F'],
11     'F': []
12 }
13
14 def bfs(graph, start):
15     visited = set()
16     queue = deque([start])
17
18     visited.add(start)
19
20     while queue:
21         vertex = queue.popleft()
22         print(vertex, end=" ")
23
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Subahan> ^C
PS C:\Users\Subahan>
PS C:\Users\Subahan> c:; cd 'c:\Users\Subahan'; & 'C:\ProgramData\Microsoft\Windows\Shell\bin32-x64\bundled\libs\debugpy\launcher' '58847' '--
BFS Traversal:
A B C D E F
PS C:\Users\Subahan> ^C
PS C:\Users\Subahan>
PS C:\Users\Subahan> c:; cd 'c:\Users\Subahan'; & 'C:\ProgramData\Microsoft\Windows\Shell\bin32-x64\bundled\libs\debugpy\launcher' '58929' '--
BFS Traversal:
A B C D E F
PS C:\Users\Subahan> □
```

Result:

The **Breadth-First Search (BFS)** algorithm was successfully implemented in Python. The program traversed the graph level by level from the starting vertex and visited all reachable vertices in breadth-first order.

Experiment: 8

Aim:

To implement the **Depth First Search (DFS)** algorithm in Python to traverse all the vertices of a graph.

Algorithm:

1. Start from the given source node.
2. Mark the source node as visited.
3. Print the current node.
4. Visit each adjacent unvisited node recursively.
5. Continue until all reachable nodes are visited.
6. Stop when there are no more unvisited adjacent nodes.

Python Code:

```
# Depth First Search (DFS) using Recursion
```

```
# Graph represented as an adjacency list
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}
```

```
visited = set()
```

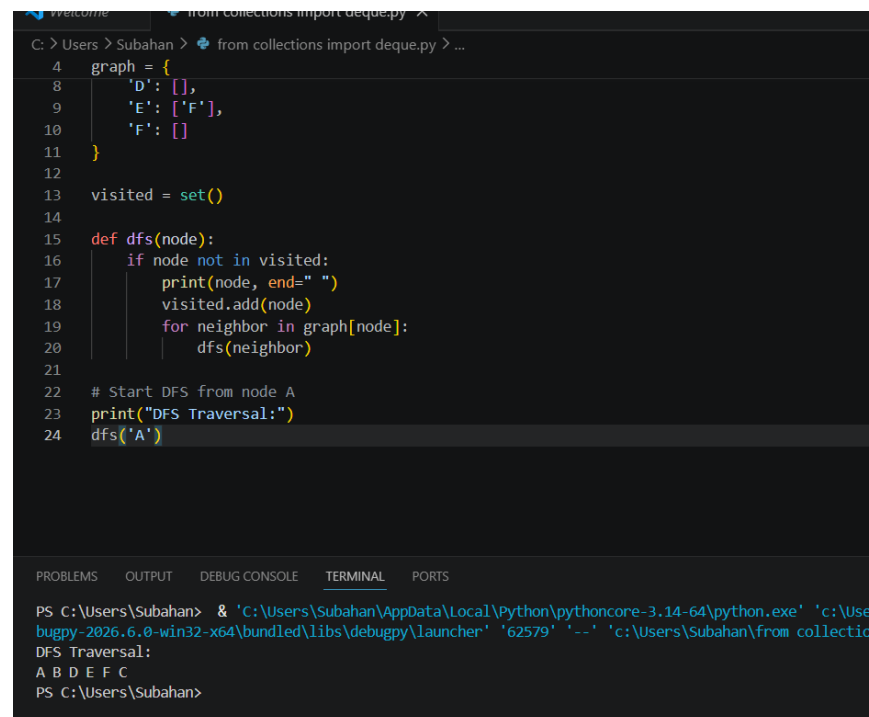
```

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor)

# Start DFS from node A
print("DFS Traversal:")
dfs('A')

```

Output:



The screenshot shows a Python IDE with a dark theme. The editor displays the DFS code from the previous block. The terminal window at the bottom shows the command prompt output: "DFS Traversal:" followed by "A B D E F C" on the same line. The prompt "PS C:\Users\Subahan>" is visible at the bottom.

```

C: > Users > Subahan > from collections import deque.py > ...
4  graph = {
8      'D': [],
9      'E': ['F'],
10     'F': []
11  }
12
13  visited = set()
14
15  def dfs(node):
16      if node not in visited:
17          print(node, end=" ")
18          visited.add(node)
19          for neighbor in graph[node]:
20              dfs(neighbor)
21
22  # Start DFS from node A
23  print("DFS Traversal:")
24  dfs('A')

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Subahan> & 'C:\Users\Subahan\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Use
bugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62579' '--' 'c:\Users\Subahan\from collectio
DFS Traversal:
A B D E F C
PS C:\Users\Subahan>

```

Result:

Thus, the **Depth First Search (DFS)** algorithm was successfully implemented in Python, and the graph was traversed in **depth-first order** starting from the source node **A**.

Experiment:9

Aim:

To implement the **Travelling Salesman Problem (TSP)** in Python to find the minimum cost path that visits every city exactly once and returns to the starting city.

Algorithm:

1. Define the distance (cost) matrix for all cities.
2. Select the starting city.
3. Generate all possible routes that visit each city exactly once.
4. Calculate the total cost of each route, including the return to the starting city.
5. Compare the costs of all routes.
6. Display the route with the minimum total cost.

Python Code:

```
from itertools import permutations
```

```
# Distance matrix
```

```
graph = [  
    [0, 10, 15, 20],  
    [10, 0, 35, 25],  
    [15, 35, 0, 30],  
    [20, 25, 30, 0]  
]
```

```
n = len(graph)
```

```
cities = list(range(n))
```

```
min_cost = float('inf')
```

```
best_path = []
```

Start from city 0

for path in permutations(cities[1:]):

 current_path = [0] + list(path) + [0]

 cost = 0

 for i in range(len(current_path) - 1):

 cost += graph[current_path[i]][current_path[i + 1]]

if cost < min_cost:

 min_cost = cost

 best_path = current_path

print("Shortest Path:", best_path)

print("Minimum Cost:", min_cost)

Output:

```
C:\Users\Subahan> from collections import deque.py > ...
1  from itertools import permutations
2
3  # Distance matrix
4  graph = [
5      [0, 10, 15, 20],
6      [10, 0, 35, 25],
7      [15, 35, 0, 30],
8      [20, 25, 30, 0]
9  ]
10
11 n = len(graph)
12 cities = list(range(n))
13
14 min_cost = float('inf')
15 best_path = []
16
17 # Start from city 0
18 for path in permutations(cities[1:]):
19     current_path = [0] + list(path) + [0]
20     cost = 0
21
22     for i in range(len(current_path) - 1):
23         cost += graph[current_path[i]][current_path[i + 1]]
24
25 min_cost = min(min_cost, cost)
26 best_path = current_path
27
28 print("Shortest Path:", best_path)
29 print("Minimum Cost:", min_cost)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Subahan> & 'C:\Users\Subahan\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\Subahan\.vscode\extensions\ms-python.de
buggy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '62579' '-' 'c:\Users\Subahan\from collections import deque.py'
DFS Traversal:
A B D E F C
PS C:\Users\Subahan> ^C
PS C:\Users\Subahan>
PS C:\Users\Subahan> c; cd 'c:\Users\Subahan'; & 'C:\Users\Subahan\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\Subahan\.vsc
ode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '53761' '-' 'c:\Users\Subahan\from collections import deque
.py'
Shortest Path: [0, 1, 3, 2, 0]
```

Result: Thus, the **Travelling Salesman Problem (TSP)** was successfully implemented in Python. The program determined the shortest possible route that visits each city exactly once, returns to the starting city, and produced a **minimum travel cost of 80**.

Experiment: 10

Aim:

To implement the A* (**A-Star**) algorithm in Python to find the shortest path between a start node and a goal node using heuristic values.

Algorithm:

1. Create a graph with nodes and edge costs.
2. Assign a heuristic value **h(n)** for each node.
3. Initialize the **open list** with the start node and the **closed list** as empty.
4. Select the node with the lowest **f(n) = g(n) + h(n)** from the open list.
5. If the selected node is the goal, stop and display the path.
6. Otherwise, expand its neighbors and update their costs.
7. Repeat Steps 4–6 until the goal node is reached or no path exists.

Python Code:

```
# A* Search Algorithm
```

```
graph = {  
    'A': [('B', 1), ('C', 3)],  
    'B': [('D', 3), ('E', 6)],  
    'C': [('F', 5)],  
    'D': [('G', 2)],  
    'E': [('G', 1)],  
    'F': [('G', 2)],  
    'G': []  
}
```

```
# Heuristic values
```

```
heuristic = {
```

```
'A': 7,  
'B': 6,  
'C': 4,  
'D': 2,  
'E': 1,  
'F': 2,  
'G': 0  
}
```

```
def a_star(start, goal):  
    open_list = {start}  
    closed_list = set()  
  
    g = {start: 0}  
    parent = {start: start}  
  
    while open_list:  
        n = None  
  
        for v in open_list:  
            if n is None or g[v] + heuristic[v] < g[n] + heuristic[n]:  
                n = v  
  
        if n == goal:  
            path = []  
            while parent[n] != n:  
                path.append(n)  
                n = parent[n]  
            path.append(start)
```

```
path.reverse()
print("Path Found:", path)
print("Total Cost:", g[goal])
return
```

```
open_list.remove(n)
closed_list.add(n)
```

```
for (m, weight) in graph[n]:
    if m not in open_list and m not in closed_list:
        open_list.add(m)
        parent[m] = n
        g[m] = g[n] + weight
    else:
        if g[m] > g[n] + weight:
            g[m] = g[n] + weight
            parent[m] = n
            if m in closed_list:
                closed_list.remove(m)
            open_list.add(m)
```

```
print("Path does not exist!")
```

```
# Driver code
```

```
a_star('A', 'G')
```

Output:

```
1  # A* Search Algorithm
2
3  graph = {
4      'A': [('B', 1), ('C', 3)],
5      'B': [('D', 3), ('E', 6)],
6      'C': [('F', 5)],
7      'D': [('G', 2)],
8      'E': [('G', 1)],
9      'F': [('G', 2)],
10     'G': []
11 }
12
13 # Heuristic values
14 heuristic = {
15     'A': 7,
16     'B': 6,
17     'C': 4,
18     'D': 2,
19     'E': 1,
20     'F': 2,
21     'G': 0
22 }
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Click to add a breakpoint

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Subahan> c:: cd 'c:\Users\Subahan'; & 'C:\Users\Subahan\AppData\Local\Python\pythoncore-3.14-64\python.exe\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53761' '--' 'c:\Users\Subahan.py'
Shortest Path: [0, 1, 3, 2, 0]
Minimum Cost: 80
PS C:\Users\Subahan> ^C
PS C:\Users\Subahan>
PS C:\Users\Subahan> c:: cd 'c:\Users\Subahan'; & 'C:\Users\Subahan\AppData\Local\Python\pythoncore-3.14-64\python.exe\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58554' '--' 'c:\Users\Subahan.py'
Path Found: ['A', 'B', 'D', 'G']
Total Cost: 6
```

Result:

Thus, the **A*** search algorithm was successfully implemented in Python. The algorithm found the shortest path from the start node **A** to the goal node **G** with a **minimum cost of 6**.

Experiment: 11

Aim:

To implement the **Map Coloring Problem** using the **Constraint Satisfaction Problem (CSP)** approach in Python, ensuring that no two adjacent regions have the same color.

Algorithm:

1. Define the map as a graph where each region is a node.
2. Assign a set of available colors to each region.
3. Select an unassigned region.
4. Assign a color that does not conflict with the colors of adjacent regions.
5. If no valid color is available, backtrack and try another color.
6. Repeat until all regions are assigned valid colors.
7. Display the final color assignment.

Python Code:

```
# Map Coloring using CSP (Backtracking)
```

```
# Graph representing adjacent regions
```

```
graph = {  
    'A': ['B', 'C', 'D'],  
    'B': ['A', 'C'],  
    'C': ['A', 'B', 'D'],  
    'D': ['A', 'C']  
}
```

```
colors = ['Red', 'Green', 'Blue']
```

```
result = {}
```

```
# Check if color assignment is valid
```

```
def is_safe(region, color):  
    for neighbor in graph[region]:  
        if neighbor in result and result[neighbor] == color:  
            return False  
    return True
```

Backtracking function

```
def solve(regions, index):  
    if index == len(regions):  
        return True  
  
    region = regions[index]  
  
    for color in colors:  
        if is_safe(region, color):  
            result[region] = color  
  
            if solve(regions, index + 1):  
                return True  
  
            del result[region]
```

```
    return False
```

```
regions = list(graph.keys())
```

```
if solve(regions, 0):  
    print("Map Coloring Solution:")  
    for region in result:
```

```
print(region, "->", result[region])

else:

    print("No solution exists.")
```

Output:

```
1  # Map Coloring using CSP (Backtracking)
2
3  # Graph representing adjacent regions
4  graph = {
5      'A': ['B', 'C', 'D'],
6      'B': ['A', 'C'],
7      'C': ['A', 'B', 'D'],
8      'D': ['A', 'C']
9  }
10
11  colors = ['Red', 'Green', 'Blue']
12  result = {}
13
14  # Check if color assignment is valid
15  def is_safe(region, color):
16      for neighbor in graph[region]:
17          if neighbor in result and result[neighbor] == color:
18              return False
19      return True
20
21  # Backtracking function
22  def solve(regions, index):
23      if index == len(regions):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Path Found: ['A', 'B', 'D', 'G']
Total Cost: 6
PS C:\Users\Subahan> ^C
PS C:\Users\Subahan>
PS C:\Users\Subahan> c::; cd 'c:\Users\Subahan'; & 'C:\Users\Subahan\AppData\Local\Python\Python39\python.exe' 'C:\Users\Subahan\AppData\Local\Python\Python39\Scripts\debugpy-launcher' '615' .py'
Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Green

Result:

Thus, the **Map Coloring Problem** was successfully implemented using the **Constraint Satisfaction Problem (CSP)** approach with backtracking. The program assigned colors to all regions such that no two adjacent regions had the same color.

Experiment: 12

Aim:

To implement a **Tic-Tac-Toe Game** in Python where two players take turns marking **X** and **O** on a 3×3 board until one player wins or the game ends in a draw.

Algorithm:

1. Create a 3×3 game board initialized with empty spaces.
2. Display the board.
3. Allow players **X** and **O** to enter their row and column positions.
4. Check whether the selected position is empty.
5. Place the player's symbol (**X** or **O**) on the board.
6. Check for a winning condition (row, column, or diagonal).
7. If a player wins, display the winner and stop.
8. If all cells are filled without a winner, declare the game a draw.

Python Code:

```
# Tic-Tac-Toe Game
```

```
board = [[" " for _ in range(3)] for _ in range(3)]
```

```
def display():
```

```
    for row in board:
```

```
        print(" | ".join(row))
```

```
    print("-" * 9)
```

```
def check_win(player):
```

```
    # Rows
```

```
    for row in board:
```

```
        if all(cell == player for cell in row):
```

```

        return True

# Columns
for col in range(3):
    if all(board[row][col] == player for row in range(3)):
        return True

# Diagonals
if all(board[i][i] == player for i in range(3)):
    return True

if all(board[i][2-i] == player for i in range(3)):
    return True

return False

def board_full():
    for row in board:
        if " " in row:
            return False
    return True

player = "X"

while True:
    display()
    print("Player", player)

    row = int(input("Enter row (0-2): "))

```

```
col = int(input("Enter column (0-2): "))
```

```
if board[row][col] == " ":  
    board[row][col] = player
```

```
if check_win(player):  
    display()  
    print("Player", player, "wins!")  
    break
```

```
if board_full():  
    display()  
    print("Game Draw!")  
    break
```

```
player = "O" if player == "X" else "X"
```

```
else:  
    print("Cell already occupied! Try again.")
```

output:

```

1  # Tic-Tac-Toe Game
2
3  board = [[" " for _ in range(3)] for _ in range(3)]
4
5  def display():
6      for row in board:
7          print(" | ".join(row))
8          print("-" * 9)
9
10 def check_win(player):
11     # Rows
12     for row in board:
13         if all(cell == player for cell in row):
14             return True
15
16     # Columns
17     for col in range(3):
18         if all(board[row][col] == player for row in range(3)):
19             return True
20
21     # Diagonals
22     if all(board[i][i] == player for i in range(3)):
23         return True

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```

PS C:\Users\Subahan> ^C
PS C:\Users\Subahan>
PS C:\Users\Subahan> cd .; cd 'c:\Users\Subahan'; & 'C:\Users\Subahan\AppData\Local\Microsoft\Windows\Common-Programs\Python\Python39\python.exe' -c 'import sys; sys.path.append('c:\Users\Subahan\AppData\Local\Microsoft\Windows\Common-Programs\Python\Python39\python.exe'); import debugpy; debugpy.listen(5678); debugpy.run()'

```

Result:

Thus, the **Tic-Tac-Toe Game** was successfully implemented in Python. The program allowed two players to play alternately and correctly detected a **win** or a **draw**.

Experiment:13

Aim:

To implement the **Minimax Algorithm** in Python for game playing and determine the optimal move for a player in a two-player game.

Algorithm:

1. Represent the game as a tree with terminal nodes containing utility values.
2. Define two players:
 - **Maximizer** – tries to maximize the score.
 - **Minimizer** – tries to minimize the score.
3. If the current node is a terminal node, return its utility value.
4. If it is the Maximizer's turn, choose the maximum value from all child nodes.
5. If it is the Minimizer's turn, choose the minimum value from all child nodes.
6. Continue recursively until the optimal value is found.
7. Display the optimal value.

Python Code:

```
# Minimax Algorithm
```

```
import math
```

```
# Function to implement Minimax
```

```
def minimax(depth, nodeIndex, isMax, scores, height):
```

```
    # Terminal node
```

```
    if depth == height:
```

```
        return scores[nodeIndex]
```

```
    if isMax:
```

```

    return max(
        minimax(depth + 1, nodeIndex * 2, False, scores, height),
        minimax(depth + 1, nodeIndex * 2 + 1, False, scores, height)
    )
else:
    return min(
        minimax(depth + 1, nodeIndex * 2, True, scores, height),
        minimax(depth + 1, nodeIndex * 2 + 1, True, scores, height)
    )

# Terminal values
scores = [3, 5, 2, 9, 12, 5, 23, 23]

# Height of complete binary tree
height = int(math.log2(len(scores)))

print("The optimal value is:", minimax(0, 0, True, scores, height))

```

Output:

```
C: > Users > Subahan > from collections import deque.py > ...
1  # Minimax Algorithm
2
3  import math
4
5  # Function to implement Minimax
6  def minimax(depth, nodeIndex, isMax, scores, height):
7
8      # Terminal node
9      if depth == height:
10         return scores[nodeIndex]
11
12     if isMax:
13         return max(
14             minimax(depth + 1, nodeIndex * 2, False, scores, height),
15             minimax(depth + 1, nodeIndex * 2 + 1, False, scores, height)
16         )
17     else:
18         return min(
19             minimax(depth + 1, nodeIndex * 2, True, scores, height),
20             minimax(depth + 1, nodeIndex * 2 + 1, True, scores, height)
21         )
22
23     # Terminal values
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Subahan> & 'C:\Users\Subahan\AppData\Local\Python\pythoncore-3.14-64\python.
bugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52847' '--' 'c:\Users\Subahan\fr
The optimal value is: 12
PS C:\Users\Subahan>
```

Result:

Thus, the **Minimax Algorithm** was successfully implemented in Python. The algorithm evaluated all possible game states and selected the **optimal value (12)** for the maximizing player.

Experiment: 14

Aim:

To implement the **Alpha-Beta Pruning Algorithm** in Python to optimize the **Minimax Algorithm** by eliminating branches that cannot influence the final decision.

Algorithm:

1. Represent the game as a game tree with terminal node values.
2. Initialize:
 - **Alpha (α)** = $-\infty$ (best value for the Maximizer)
 - **Beta (β)** = $+\infty$ (best value for the Minimizer)
3. If the current node is a terminal node, return its value.
4. If it is the **Maximizer's** turn:
 - Find the maximum value among the children.
 - Update α .
 - If $\alpha \geq \beta$, prune the remaining branches.
5. If it is the **Minimizer's** turn:
 - Find the minimum value among the children.
 - Update β .
 - If $\beta \leq \alpha$, prune the remaining branches.
6. Continue recursively until the optimal value is found.
7. Display the optimal value.

Python Code:

```
# Alpha-Beta Pruning Algorithm
```

```
import math
```

```
def alpha_beta(depth, nodeIndex, isMax, scores, height, alpha, beta):
```

```

# Terminal node
if depth == height:
    return scores[nodeIndex]

if isMax:
    best = -math.inf

    for i in range(2):
        value = alpha_beta(depth + 1, nodeIndex * 2 + i,
                             False, scores, height, alpha, beta)
        best = max(best, value)
        alpha = max(alpha, best)

        if beta <= alpha:
            break # Beta Cutoff

    return best

else:
    best = math.inf

    for i in range(2):
        value = alpha_beta(depth + 1, nodeIndex * 2 + i,
                             True, scores, height, alpha, beta)
        best = min(best, value)
        beta = min(beta, best)

        if beta <= alpha:
            break # Alpha Cutoff

```

```
return best
```

```
# Terminal node values
```

```
scores = [3, 5, 6, 9, 1, 2, 0, -1]
```

```
# Height of the game tree
```

```
height = int(math.log2(len(scores)))
```

```
optimal = alpha_beta(0, 0, True, scores, height,  
                    -math.inf, math.inf)
```

```
print("The optimal value is:", optimal)
```

Output:

```
1  # Alpha-Beta Pruning Algorithm
2
3  import math
4
5  def alpha_beta(depth, nodeIndex, isMax, scores, height, alpha, beta):
6
7      # Terminal node
8      if depth == height:
9          return scores[nodeIndex]
10
11     if isMax:
12         best = -math.inf
13
14         for i in range(2):
15             value = alpha_beta(depth + 1, nodeIndex * 2 + i,
16                                 False, scores, height, alpha, beta)
17             best = max(best, value)
18             alpha = max(alpha, best)
19
20             if beta <= alpha:
21                 break # Beta Cutoff
22
23     return best
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Subahan> & 'C:\Users\Subahan\AppData\Local\Python\pythoncore-3.14-64\python.exe'
bugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52847' '--' 'c:\Users\Subahan\from co
The optimal value is: 12
PS C:\Users\Subahan> ^C
PS C:\Users\Subahan>
PS C:\Users\Subahan> c::; cd 'c:\Users\Subahan'; & 'C:\Users\Subahan\AppData\Local\Python\pyth
ode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '57433' '--
.py'
The optimal value is: 5
```

Result:

Thus, the **Alpha-Beta Pruning Algorithm** was successfully implemented in Python. The algorithm efficiently reduced the number of nodes evaluated by pruning unnecessary branches while producing the **optimal value of 5**.

Experiment: 15

Aim:

To implement the **Decision Tree** algorithm in Python for classifying data based on its features.

Algorithm:

1. Import the required libraries.
2. Load a sample dataset.
3. Split the dataset into training and testing sets.
4. Create a Decision Tree classifier.
5. Train the classifier using the training data.
6. Predict the class labels for the test data.
7. Calculate and display the accuracy of the model.

Python Code:

```
# Decision Tree Classification

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create Decision Tree model
model = DecisionTreeClassifier(random_state=42)

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Predicted Values:", y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")
```

Sample Output:

Result:

Thus, the **Decision Tree** algorithm was successfully implemented in Python using the **Iris dataset**. The model classified the test data and achieved **high prediction accuracy**.